G Prashanth

192311366

CSA1709-ARTIFICIAL INTELLIGENCE

---

13. Minimax Algorithm for Gaming

Aim

To implement the Minimax algorithm for game playing in Python.

Algorithm

1. Create a game tree with terminal node values.
2. If the node is a leaf, return its value.
3. If it is the maximizing player's turn, choose the maximum value.
4. If it is the minimizing player's turn, choose the minimum value.
5. Continue recursively until the optimal move is found.
6. Display the optimal value.

Program

```
import math
def minimax(depth, nodeIndex, maximizingPlayer, values, maxDepth):
    if depth == maxDepth:
        return values[nodeIndex]
    if maximizingPlayer:
        return max(
            minimax(depth + 1, nodeIndex * 2, False, values, maxDepth),
            minimax(depth + 1, nodeIndex * 2 + 1, False, values, maxDepth)
        )
    else:
        return min(
            minimax(depth + 1, nodeIndex * 2, True, values, maxDepth),
            minimax(depth + 1, nodeIndex * 2 + 1, True, values, maxDepth)
        )
```

```
values = [3, 5, 2, 9, 12, 5, 23, 23]
print("Optimal Value:", minimax(0, 0, True, values, 3))
```

Sample Input

Terminal Node Values:

3 5 2 9 12 5 23 23

Sample Output

Optimal Value: 12

Result

Thus, the Minimax algorithm was implemented successfully.

---

14. Alpha-Beta Pruning Algorithm

Aim

To implement the Alpha-Beta Pruning algorithm in Python.

Algorithm

1. Build the game tree.
2. Initialize Alpha = -∞ and Beta = +∞.
3. Traverse the tree using Minimax.
4. Update Alpha for maximizing nodes.
5. Update Beta for minimizing nodes.
6. Prune branches when Alpha ≥ Beta.
7. Display the optimal value.

Program

```
import math
MAX = 1000
MIN = -1000
def alphabeta(depth, nodeIndex, maximizingPlayer,
              values, alpha, beta, maxDepth):
```

```
if depth == maxDepth:
    return values[nodeIndex]
if maximizingPlayer:
    best = MIN
    for i in range(2):
        val = alphabeta(depth + 1,
                        nodeIndex * 2 + i,
                        False,
                        values,
                        alpha,
                        beta,
                        maxDepth)
        best = max(best, val)
        alpha = max(alpha, best)
        if beta <= alpha:
            break
    return best
else:
    best = MAX
    for i in range(2):
        val = alphabeta(depth + 1,
                        nodeIndex * 2 + i,
                        True,
                        values,
                        alpha,
                        beta,
                        maxDepth)
        best = min(best, val)
        beta = min(beta, best)
```

```
            if beta <= alpha:
                break
        return best
values = [3, 5, 6, 9, 1, 2, 0, -1]

print("Optimal Value:",
      alphabeta(0, 0, True, values, MIN, MAX, 3))
```

Sample Input

Leaf Values:

3 5 6 9 1 2 0 -1

Sample Output

Optimal Value: 5

Result

Thus, the Alpha-Beta Pruning algorithm was implemented successfully.

---

15. Decision Tree

Aim

To implement a Decision Tree classifier using Python.

Algorithm

1. Import the required libraries.
2. Load the dataset.
3. Split the dataset into training and testing sets.
4. Train the Decision Tree classifier.
5. Predict the test data.
6. Calculate and display the accuracy.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data,
    iris.target,
    test_size=0.3,
    random_state=1
)
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, prediction))
```

Sample Input

Dataset: Iris Dataset

Sample Output

Accuracy: 0.97

Result

Thus, the Decision Tree classifier was implemented successfully.

---

16. Feed Forward Neural Network

Aim

To implement a Feed Forward Neural Network using Python.

Algorithm

1. Import the required libraries.
2. Load the dataset.
3. Split the dataset into training and testing sets.
4. Create a Feed Forward Neural Network.

5. Train the model using the training data.
6. Predict the output for the test data.
7. Calculate and display the accuracy.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data,
    iris.target,
    test_size=0.3,
    random_state=1
)
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=1000,
    random_state=1
)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, prediction))
```

Sample Input

Dataset: Iris Dataset

Sample Output

Accuracy: 0.98

Result

Thus, the Feed Forward Neural Network was implemented successfully.